

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPD801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

28th May 2026



Abstract

Words words words words words words words words words
Ord ord ord ord ord ord ord ord ord ord

CONTENTS

A	Code Availability	3
B	Lean File Structure	3
C	Common Lean Notation	3
D	Selected Core Definitions	4
D.1	Session Types	4
D.2	Processes	5
D.3	Judgements	6
D.4	Transition Labels	8
D.5	Transition Systems	9
E	Erasure Functions and Main Theorems	13
F	Selected Proof Infrastructure	13

A Code Availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: <https://github.com/MikkelBrixNielsen/LeanPiLL>

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions are available in the repository history. These earlier versions are not part of the submitted artefact.

The appendix contains selected excerpts of the development. These excerpts are typeset to closely reflect the corresponding Lean code. In particular, some Unicode notation and spacing is rendered typographically rather than shown as copy-pasteable Lean source. The complete copy-pasteable Lean files, including all definitions, auxiliary lemmas, and proof scripts, are available in the accompanying codebase.

B Lean File Structure

```
PiLL/
  Model/
    Environments/
    HyperEnvironments/
    Judgement/
    Processes/
    STypes/
    Base.lean
  Semantics/
    EnvStep/
    JudgementStep/
    ProcStep/
    Labels.lean
  SessionFidelity/
    ProcStep.lean
    EnvStep.lean
    Typability.lean
    SubjectReduction.lean
```

Listing 1. Top-level structure of the Lean development.

C Common Lean Notation

```
class HasShift (Subject : Type) where
  shift : Subject -> Nat -> Nat -> Subject

notation:max S:max " ↑ " d:max ", " c:max => HasShift.shift S d c
notation:max S:max " ↑ " c:max => HasShift.shift S 0 c
notation:max S:max "+" => HasShift.shift S 0 1

class HasSubst (Subject Replacement Target : Type) where
  subst : Subject -> Replacement -> Target -> Subject

notation:max S "{" R " //" T "}" => HasSubst.subst S R T

class HasOpen (Subject Replacement Target : Type) where
  open_ : Subject -> Replacement -> Target -> Subject
```

```

148
149 notation: max S: max "⟨( " T " | " R ⟩)" => HasOpen.open_ S R T
150 notation: max S: max "⟨( " R ⟩)" => HasOpen.open_ S R 0
151
152 class HasClose (Subject Replacement Target : Type) where
153   close_ : Subject -> Replacement -> Target -> Subject
154
155 notation: max S: max "⟨⟨ " T " | " R ⟩⟩" => HasClose.close_ S R T
156 notation: max S: max "⟨⟨ " R ⟩⟩" => HasClose.close_ S R 0
157
158 class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
159   open_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
160
161 notation: max S: max "⟨( " T " | " R1 ", " R2 ⟩)" => HasOpenTwo.open_ S R1 R2 T
162 notation: max S: max "⟨( " R1 ", " R2 ⟩)" => HasOpenTwo.open_ S R1 R2 0
163
164 class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
165   close_ : Subject -> Replacement_1 -> Replacement_2 -> Target -> Subject
166
167 notation: max S: max "⟨⟨ " T " | " R1 ", " R2 ⟩⟩" => HasCloseTwo.close_ S R1 R2 T
168 notation: max S: max "⟨⟨ " R1 ", " R2 ⟩⟩" => HasCloseTwo.close_ S R1 R2 0
169
170 class HasBracket (Subject Content Result : Type) where
171   brack : Subject -> Content -> Result
172
173 class HasParen (Subject Content Result : Type) where
174   paren : Subject -> Content -> Result
175
176 notation: 80 x "[]" => HasBracket.brack x ()
177 notation: 80 x "["y"]" => HasBracket.brack x y
178 notation: 80 x "()" => HasParen.paren x ()
179 notation: 80 x "( y )" => HasParen.paren x y
180
181 class HasPerm ( $\alpha$  : Type) where
182   perm :  $\alpha$  ->  $\alpha$  -> Prop
183
184 infixr:54 " ~ " => HasPerm.perm

```

Listing 2. Common typeclasses and notation used throughout the Lean development. These abbreviations are used uniformly for shifting, substitution, opening, closing, prefix notation, and permutation.

D Selected Core Definitions

D.1 Session Types

```

189 abbrev Atom := Nat
190 abbrev FTVar := Nat
191 abbrev BTVar := Nat
192
193 inductive TVar : Type
194   | free   (x : FTVar)
195   | bound  (x : BTVar)

```

```

197 deriving DecidableEq, Repr
198
199 inductive Types : Type where
200   | atom      (a : Atom)
201   | atomDual  (a : Atom)
202   | var       (v : TVar)
203   | varDual   (v : TVar)
204   | one
205   | bot
206   | tensor    (A B : Types)
207   | parr      (A B : Types)
208   | oplus     (A B : Types)
209   | amp       (A B : Types)
210   | bang      (A : Types)
211   | quest     (A : Types)
212   | forall_   (A : Types)
213   | exists_   (A : Types)
214 deriving DecidableEq, BEq, Repr

```

Listing 3. Locally nameless representation of type variables and the Lean definition of session types. Free type variables are represented explicitly, while bound type variables are represented by De Bruijn indices.

D.2 Processes

This appendix gives the core Lean representation of channels and process syntax. Channels are represented using the locally nameless approach discussed in ??, separating free names from bound indices.

```

223 abbrev Fpname := Nat
224 abbrev Bpname := Nat
225
226 inductive Channel : Type where
227   | free    (x : Fpname)
228   | bound   (x : Bpname)
229 deriving DecidableEq, BEq, Repr
230
231 prefix:max "#" => Channel.free
232 prefix:max "$" => Channel.bound
233
234 inductive Proc : Type where
235   | nil
236   | one      (x : Channel) (P : Proc)
237   | bot      (x : Channel) (P : Proc)
238   | tensor   (x : Channel) (P : Proc)
239   | parr     (x : Channel) (P : Proc)
240   | cut      (P : Proc)
241   | par      (P Q : Proc)
242   | selectL  (x : Channel) (P : Proc)
243   | selectR  (x : Channel) (P : Proc)
244   | amp      (x : Channel) (P Q : Proc)
245   | output   (x : Channel) (P : Proc) (A : Types)
246   | input    (x : Channel) (P : Proc)

```

```

246 | server      (x : Channel) (zs : Finset Channel) (P : Proc)
247 | consume    (x : Channel) (P : Proc)
248 | duplicate   (x : Channel) (P : Proc)
249 | dispose    (x : Channel) (P : Proc)
250 | link       (x y : Channel)
251 deriving DecidableEq, BEq

```

Listing 4. Locally nameless representation of channels and process syntax. Free channels are represented explicitly, while bound channels are represented by De Bruijn indices.

D.3 Judgements

```

258 inductive Typing : Nat -> Proc -> HyperEnv -> Prop where
259 | mix0 {n : Nat} :
260   Typing n 0 ∅
261
262 | mix {G H : HyperEnv} {P Q : Proc} {n : Nat} (hD : G.disjoint H) :
263   Typing n P G -> Typing n Q H ->
264   Typing n (P |p Q) (G |h H)
265
266 | one {P : Proc} {x : FPNName} {n : Nat} :
267   Typing n P ∅ ->
268   Typing n (#x[[]].P) [[x:1]]
269
270 | bot {Γ : Env} {P : Proc} {x : FPNName} {n : Nat} (hF : x ∉ Γ.names) :
271   Typing n P [Γ] ->
272   Typing n (#x[()].P) [x:⊥::Γ]
273
274 | cut {G : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat}
275   (L : Finset FPNName) :
276   (∀ x ∉ L, ∀ y ∉ L, x ≠ y ->
277     Typing n (P(#x, #y)) (G |h [x:A::Γ] |h [y:A⊥::Δ])) ->
278   Typing n (v(•,•) P) (G |h [Γ, Δ])
279
280 | tensor {Γ Δ : Env} {P : Proc} {x : FPNName} {A B : Types} {n : Nat}
281   (hF : x ∉ Γ.names ∧ x ∉ Δ.names) (L : Finset FPNName) :
282   (∀ y ∉ L, Typing n (P(#y)) ([y:A::Γ] |h [x:B::Δ])) ->
283   Typing n (#x[•].P) [x:A ⊗ B::Γ, Δ]
284
285 | parr {Γ : Env} {P : Proc} {x : FPNName} {A B : Types}
286   {n : Nat} (hF : x ∉ Γ.names) (L : Finset FPNName) :
287   (∀ y ∉ L, Typing n (P(#y)) [y:A::x:B::Γ]) ->
288   Typing n (#x[•].P) [x:A ∘ B::Γ]
289
290 | oplus1 {Γ : Env} {P : Proc} {x : FPNName} {A B : Types} {n : Nat} :
291   B.lc n -> Typing n P [x:A::Γ] ->
292   Typing n (#x[[]L].P) [x:A ⊕ B::Γ]
293
294 | oplus2 {Γ : Env} {P : Proc} {x : FPNName} {A B : Types} {n : Nat} :
295   A.lc n -> Typing n P [x:B::Γ] ->
296   Typing n (#x[[]R].P) [x:A ⊕ B::Γ]

```

```

295 | amp {Γ : Env} {P Q : Proc} {x : FPName} {A B : Types} {n : Nat} :
296   Typing n P [x:A::Γ] -> Typing n Q [x:B::Γ] ->
297   Typing n (#x.case{L : P, R : Q}) [x:A & B::Γ]
298
299 | quest {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
300   Typing n P [x:A::Γ] ->
301   Typing n (#x[[USE]].P) [x:??A::Γ]
302
303 | bang {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
304   ?_eΓ -> Typing n P [x:A::Γ] ->
305   Typing n (!#x<Γ.names.image Channel.free>.{P}) [x:!!A::Γ]
306
307 | w {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
308   (hF : x ∉ Γ.names) :
309   A.lc n -> Typing n P [Γ] ->
310   Typing n (#x[[DISP]].P) [x:??A::Γ]
311
312 | c {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
313   (hF : x ∉ Γ.names) (L : Finset FPName) :
314   (∀ y ∉ L, Typing n P((#y)) [x:??A::y:??A::Γ]) ->
315   Typing n (#x[[DUP]]((●)).P) [x:??A::Γ]
316
317 | exists_ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
318   A.lc n -> Typing n P [x:B{A // 0}::Γ] ->
319   Typing n (#x[[A]].P) [x:∃.B::Γ]
320
321 | forall_ {Γ : Env} {P : Proc} {x : FPName} {B : Types} {n : Nat} :
322   Typing (n + 1) P [x:B::Γ+] ->
323   Typing n (#x((◊)).P) [x:∀.B::Γ]
324
325 | ax {x y : FPName} {A : Types} {n : Nat} (hneq : x ≠ y) :
326   A.lc n ->
327   Typing n (#x ↔p #y) [x:A⊥:: [y:A]]
328
329 | exchange_env {G : HyperEnv} {Γ Δ : Env} {P : Proc} {n : Nat} :
330   Typing n P (Γ::G) -> Γ ~ Δ ->
331   Typing n P (Δ::G)
332
333 | exchange_hyper {G H : HyperEnv} {P : Proc} {n : Nat} :
334   Typing n P G -> G ~ H ->
335   Typing n P H
336
337 notation:50 n " ⊢ " P " :: " G => Typing n P G

```

Listing 5. The main typing judgement. The judgement `Typing n P G` states that process `P` is well typed under hyperenvironment `G` at type-variable depth `n`.

The constructors in the above listing cover the full static π LL typing relation. The multiplicative binders `cut`, `tensor`, and `parr` use co-finite premises, while `exchange_env` and `exchange_hyper`

internalise exchange for environments and hyperenvironments. These exchange constructors are used throughout the metatheory to compensate for the list-based representation of contexts.

```

def proc {G : HyperEnv} {P : Proc} {n : Nat}
  ( _ : n ⊢ P :: G ) : Proc := P

def env {G : HyperEnv} {P : Proc} {n : Nat}
  ( _ : n ⊢ P :: G ) : HyperEnv := G

```

Listing 6. Projection functions from typing derivations. Since the judgement is indexed by both the process and hyperenvironment, these projections simply recover the corresponding indices.

D.4 Transition Labels

This appendix gives the full Lean definitions of the label and action datatypes discussed in ??, together with the associated free-name, introduced-name, and well-formedness functions used by the operational semantics.

```

inductive Mu : Type
| L
| R
| DISP
| DUP
| USE
deriving Repr, DecidableEq

inductive Act : Type
| one      (x : FPNName)
| bot      (x : FPNName)
| tensor   (x y : FPNName)
| parr     (x y : FPNName)
| output   (x : FPNName) (A : Types)
| input    (x : FPNName) (A : Types)
| muBrack  (x : FPNName) (μ : Mu)
| muParen  (x : FPNName) (μ : Mu)
deriving Repr, DecidableEq

inductive Lbl : Type
| tau
| link (x y : FPNName)
| act  (p : Act)
| par  (l l' : Act)
deriving Repr, DecidableEq

@[simp] def fNamesAct : Act -> Finset FPNName -- free names
| .one x | .bot x
| .tensor x _ | .parr x _
| .input x _ | .output x _
| .muBrack x _ | .muParen x _ => {x}

@[simp] def iNamesAct : Act -> Finset FPNName -- introduced names
| .one _ | .bot _ | .input _ _ | .output _ _

```

```

393 | Act.muBrack _ _ | Act.muParen _ _ => {}
394 | .tensor _ y | .parr _ y => {y}
395
396 @[simp] def Lbl.f : Lbl -> Finset FPNName
397 | .tau      => {}
398 | .link x y => {x, y}
399 | .act a    => fNamesAct a
400 | .par l l' => fNamesAct l ∪ fNamesAct l'
401
402 @[simp] def Lbl.i : Lbl -> Finset FPNName
403 | .tau      => {}
404 | .link _ _ => {}
405 | .act a    => iNamesAct a
406 | .par l l' => iNamesAct l ∪ iNamesAct l'
407
408 @[simp] def Lbl.WF : Lbl -> Prop
409 | .tau => True
410 | .link _ _ => True
411 | .act _ => True
412 | .par l l' => (iNamesAct l) ∩ (iNamesAct l') = {}

```

Listing 7. Definitions of transition labels, actions, and their associated free and introduced names.

D.5 Transition Systems

This appendix collects the full inductive definitions of the three transition relations for π MLL, as referenced from ???. The `tensor_parr` and `res` constructors of `TypingStepm` are elided for length; their full definitions are available in the accompanying source archive.

```

420 inductive TypingStepm :
421   {n : Nat} -> {G : HyperEnv} -> {P : Proc} -> Typing n P G -> Lbl ->
422   {n' : Nat} -> {G' : HyperEnv} -> {P' : Proc} -> Typing n' P' G' -> Prop where
423
424 | one {P : Proc} {x : FPNName} {n : Nat} {D : Typing n P {} :
425   TypingStepm (Typing.one (x := x) D) (x[]) D
426
427 | bot {Γ : Env} {P : Proc} {x : FPNName} {n : Nat}
428   {hF : x ∉ Γ.names} {D : Typing n P [Γ]} :
429   TypingStepm (Typing.bot (x := x) hF D) (x()) D
430
431 | tensor {Γ Δ : Env} {P : Proc} {x y : FPNName} {A B : Types} {n : Nat}
432   {hF : x ∉ Γ.names ∧ x ∉ Δ.names} {L : Finset FPNName}
433   {huniq : ∀ z, z ∉ L -> Typing n (P((#z))) ([z:A::Γ] |h [x:B::Δ])}
434   {hy : y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names)} :
435   TypingStepm (Typing.tensor hF L huniq) (x|| y) (Typing_tensor_all_fresh huniq y
436   hy)
437
438 | parr {Γ : Env} {P : Proc} {x y : FPNName} {A B : Types} {n : Nat}
439   {hF : x ∉ Γ.names} {L : Finset FPNName}
440   {huniq : ∀ z, z ∉ L -> Typing n (P((#z))) [z : A :: x : B :: Γ]}
441   {hy : y ∉ ({x} ∪ P.f ∪ Γ.names)} :

```

```

442
443 | par1 {G H G' : HyperEnv} {P Q P' : Proc} {l : Lbl} {n : Nat}
444   {hD1 : G.disjoint H} {hD2 : G'.disjoint H}
445   {D : Typing n P G} {D' : Typing n P' G'} {E : Typing n Q H}
446   (h : TypingStepm D l D') (disj : l.i ∩ Q.f = ∅) :
447   TypingStepm (Typing.mix hD1 D E) l (Typing.mix hD2 D' E)
448
449 | par2 {G H H' : HyperEnv} {P Q Q' : Proc} {l : Lbl} {n : Nat}
450   {hD1 : G.disjoint H} {hD2 : G'.disjoint H'}
451   {D : Typing n P G} {E : Typing n Q H} {E' : Typing n Q' H'}
452   (h : TypingStepm E l E') (disj : l.i ∩ P.f = ∅) :
453   TypingStepm (Typing.mix hD1 D E) l (Typing.mix hD2 D E')
454
455 | syn {G G' H H' : HyperEnv} {P P' Q Q' : Proc} {l l' : Act} {n : Nat}
456   {hD1 : G.disjoint H} {hD2 : G'.disjoint H'}
457   {D : Typing n P G} {D' : Typing n P' G'}
458   {E : Typing n Q H} {E' : Typing n Q' H'}
459   (h1 : TypingStepm D l D') (h2 : TypingStepm E l' E')
460   (disj : (l l1 l').i ∩ (P lp Q).f = ∅) (WF : (l l1 l').WF) :
461   TypingStepm (Typing.mix hD1 D E) (l l1 l') (Typing.mix hD2 D' E')
462
463 | one_bot {G : HyperEnv} {Γ : Env} {P P' : Proc} {n : Nat} {L : Finset FPNName}
464   {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
465     Typing n (P((#x, #y))) (G lh [x:1::∅] lh [y:⊥::Γ])}
466   {x y : FPNName}
467   {hx : x ∉ ({y} ∪ P.f ∪ G.names ∪ Γ.names)}
468   {hy : y ∉ (P.f ∪ G.names ∪ Γ.names)}
469   {D' : Typing n P' (G lh [∅, Γ])}
470   (hStep : TypingStepm (Typing.one_bot_all_fresh huniq x y hx hy)
471     (x l1 l1 y l1 D')) :
472   TypingStepm (Typing.cut L huniq) (τ) D'
473
474 | tensor_parr {G : HyperEnv} {Γ Δ Ξ : Env} {P P' : Proc} {A B : Types}
475   {n : Nat} {L : Finset FPNName}
476   {huniq : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
477     Typing n (P((#x, #y)))
478     (G lh [x:A⊗B::Γ, Δ] lh [y:A⊥⊗B⊥::Ξ])}
479   {huniq' : ∀ x ∉ L, ∀ y ∉ L, x ≠ y →
480     ∀ x' ∉ L, ∀ y' ∉ L, x' ≠ y' →
481     x ≠ x' → x ≠ y' → y ≠ x' → y ≠ y' →
482     Typing n (P'((2 | #x, #y))((#x', #y')))
483     (G lh [x:B::Δ] lh [x':A::Γ] lh [y':A⊥::y:B⊥::Ξ])}
484   {x y : FPNName} (hx : x ∉ L) (hy : y ∉ L) (hneq : x ≠ y)
485   {x' y' : FPNName} (hx' : x' ∉ L) (hy' : y' ∉ L) (hneq' : x' ≠ y')
486   (hxx' : x ≠ x') (hxy' : x ≠ y') (hyx' : y ≠ x') (hyy' : y ≠ y')
487   (hxp : x ∉ P.f) (hyP : y ∉ P.f) (hxp' : x' ∉ P.f) (hy'P : y' ∉ P.f)
488   (hxp' : x ∉ P'.f) (hyP' : y ∉ P'.f) (hxp'P : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
489   (hStep : TypingStepm (huniq x hx y hy hneq)
490     (x l1 l1 x' l1 y l1 y'))
491   (huniq' x hx y hy hneq x' hx' y' hy' hneq' hxx' hxy' hyx' hyy') :
492   TypingStepm (Typing.cut L huniq) (τ)
493   (Typing.double_cut_tensor_parr huniq')
494
495

```

```

491 | res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env} {P P' : Proc} {A : Types}
492   {n : Nat} {l : Lbl} {L L' : Finset FPNName}
493   {huniq : ∀ z ∉ L, ∀ w ∉ L, z ≠ w →
494     Typing n (P((#z, #w))) (G |h [z:A::Γ] |h [w:A⊥::Δ])}
495   {huniq' : ∀ z ∉ L', ∀ w ∉ L', z ≠ w →
496     Typing n (P'((#z, #w))) (G' |h [z:A::Γ'] |h [w:A⊥::Δ'])}
497   {x y : FPNName} (hneq : x ≠ y)
498   (hx_pre : x ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
499   (hy_pre : y ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
500   (hx_post : x ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
501   (hy_post : y ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
502   (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i)
503   (hStep : TypingStepm
504     (Typing_res_all_fresh huniq x y hx_pre hy_pre hneq) 1
505     (Typing_res_all_fresh huniq' x y hx_post hy_post hneq)) :
506     TypingStepm (Typing.cut L huniq) l (Typing.cut L' huniq')
507
508 | perm_env {G H : HyperEnv} {Γ Γ' : Env} {P P' : Proc} {n n' : Nat} {l : Lbl}
509   {D : n ⊢ P :: Γ :: G} {D' : n' ⊢ P' :: H} (hP1 : Γ ~ Γ')
510   (hTS : TypingStepm D l D') :
511   TypingStepm (Typing.exchange_env D hP1) l D'
512
513 | perm_hyper {G G' H H' : HyperEnv} {P P' : Proc} {n n' : Nat} {l : Lbl}
514   {D : n ⊢ P ⊢ G} {D' : n' ⊢ P' ⊢ H}
515   (hP1 : G ~ G') (hP2 : H ~ H')
516   (hTS : TypingStepm D l D') :
517   TypingStepm (Typing.exchange_hyper D hP1) l
518   (Typing.exchange_hyper D' hP2)

```

Listing 8. The typed transition relation TypingStep_m .

```

519
520 inductive EnvStepm : HyperEnv → Lbl → HyperEnv → Prop where
521 | one {x : FPNName} :
522   EnvStepm [[x : 1]] (x[[]]) ∅
523 | tensor {Γ Δ : Env} {x x' : FPNName} {A B : Types}
524   (hF : x' ∉ HyperEnv.names [x : A ⊗ B :: Γ Δ]) :
525   EnvStepm [x : A ⊗ B :: Γ Δ] (x[[ x' ]]) ([x' : A :: Γ] |h [x : B :: Δ])
526 | bot {Γ : Env} {x : FPNName} :
527   EnvStepm [x : ⊥ :: Γ] (x[()]) [Γ]
528 | parr {Γ : Env} {x x' : FPNName} {A B : Types}
529   (hF : x' ∉ HyperEnv.names [x : A ⋈ B :: Γ]) :
530   EnvStepm [x : A ⋈ B :: Γ] (x(( x' ))) [x' : A :: x : B :: Γ]
531 | par1 {G G' H : HyperEnv} {l : Lbl} :
532   EnvStepm G l G' →
533   EnvStepm (G |h H) l (G' |h H)
534 | par2 {G H H' : HyperEnv} {l : Lbl} :
535   EnvStepm H l H' →
536   EnvStepm (G |h H) l (G |h H')
537 | syn {G G' H H' : HyperEnv} {l l' : Act} :
538   EnvStepm G l G' → EnvStepm H l' H' →
539   EnvStepm (G |h H) (l |l l') (G' |h H')

```

```

540 | one_bot {G : HyperEnv} {Γ : Env} {x y : FPNName} :
541   EnvStepm (G |h [[x : 1]] |h [y : ⊥ :: Γ]) (x[[ ]] |1 y(( ))) (G |h [Γ]) ->
542   EnvStepm (G |h [Γ]) (τ) (G |h [Γ])
543 | tensor_parr {G : HyperEnv} {Γ Δ Ξ : Env} {x x' y y' : FPNName} {A B : Types} :
544   EnvStepm
545     (G |h [x : A ⊗ B :: Γ Δ] |h [y : A⊥ ⋈ B⊥ :: Ξ])
546     (x[[ x' ]] |1 y(( y' )))
547     (G |h [x : B :: Δ] |h [x' : A :: Γ] |h [y' : A⊥ :: y : B⊥ :: Ξ]) ->
548   EnvStepm (G |h [Γ Δ Ξ]) (τ) (G |h [Γ Δ Ξ])
549 | res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env}
550   {x y : FPNName} {A : Types} {l : Lbl}
551   (hFx : x ∉ l.i ∪ l.f) (hFy : y ∉ l.i ∪ l.f) :
552   EnvStepm
553     (G |h [x : A :: Γ] |h [y : A⊥ :: Δ]) l
554     (G' |h [x : A :: Γ'] |h [y : A⊥ :: Δ']) ->
555   EnvStepm (G |h [Γ Δ]) l (G' |h [Γ' Δ'])
556 | perm_env {G H : HyperEnv} {Γ Γ' : Env} {l : Lbl}
557   (hP : Γ ~ Γ') :
558   EnvStepm (Γ :: G) l H ->
559   EnvStepm (Γ' :: G) l H
560 | perm_hyper {G G' H H' : HyperEnv} {l : Lbl}
561   (hP1 : G ~ G') (hP2 : H ~ H') :
562   EnvStepm G l H ->
563   EnvStepm G' l H'

```

Listing 9. The environment-level transition relation EnvStep_m .

```

564
565 inductive ProcStepm : (P : Proc) -> Lbl -> (P' : Proc) -> Prop where
566 | one {P : Proc} {x : FPNName} :
567   ProcStepm (#x[[ ]]. P) (x[[ ]] ) P
568 | tensor {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
569   ProcStepm (#x[[•]]. P) (x[[ y ]] ) P((#y))
570 | bot {P : Proc} {x : FPNName} :
571   ProcStepm (#x(( )). P) (x(( )) ) P
572 | parr {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
573   ProcStepm (#x((•)). P) (x(( y )) ) P((#y))
574 | par1 {P P' Q : Proc} {l : Lbl} :
575   ProcStepm P l P' -> l.i ∩ Q.f = ∅ ->
576   ProcStepm (P |p Q) l (P' |p Q)
577 | par2 {P Q Q' : Proc} {l : Lbl} :
578   ProcStepm Q l Q' -> l.i ∩ P.f = ∅ ->
579   ProcStepm (P |p Q) l (P |p Q')
580 | syn {P P' Q Q' : Proc} {l l' : Act} :
581   ProcStepm P l P' -> ProcStepm Q l' Q' ->
582   (l |1 l').i ∩ (P |p Q).f = ∅ -> (l |1 l').WF ->
583   ProcStepm (P |p Q) (l |1 l') (P' |p Q')
584 | one_bot {P P' : Proc} {x y : FPNName}
585   (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
586   ProcStepm P((#x, #y)) (x[[ ]] |1 y(( ))) P' ->
587   ProcStepm (v((•,•)) P) (τ) P'
588 | tensor_parr {P P' : Proc} {x x' y y' : FPNName}
589   (hxP : x ∉ P.f) (hyP : y ∉ P.f)

```

```

589      (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
590      (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f)
591      (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
592      (hxx' : x ≠ x') (hxy : x ≠ y) (hxy' : x ≠ y')
593      (hyx' : y ≠ x') (hyy' : y ≠ y') (hx'y' : x' ≠ y') :
594      ProcStepm P((#x, #y)) (x[[ x' ]] l1 y(( y' ))) P'((2 | #x, #y))((#x', #y')) ->
595      ProcStepm (v((•,•)) P) (τ) (v((•,•)) (v((•,•)) P'))
596  | res {P P' : Proc} {l : Lbl} {x y : FPNName}
597      (hneq : x ≠ y)
598      (hx : x ∉ P.f ∪ P'.f) (hy : y ∉ P.f ∪ P'.f)
599      (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i) :
600      ProcStepm P((#x, #y)) l P'((#x, #y)) ->
601      ProcStepm (v((•,•)) P) l (v((•,•)) P')

```

Listing 10. The process-level transition relation ProcStep_m .

E Erasure Functions and Main Theorems

These things should be included in this appendix

- session_fidelity_proc
- session_fidelity_env
- typability_subject_reduction
- subject_reduction

F Selected Proof Infrastructure

These things should be included in this appendix

Freshness/co-finite bridge:

- Typing_tensor_all_fresh
- Typing_parr_all_fresh
- Typing_res_all_fresh if is discussed, else something similar

Typing inversion:

- Typing_inv_one
- Typing_inv_tensor
- maybe Typing_inv_cut

Hyperenvironment extraction/block preservation:

- one representative HyperEnv.Perm.extract_* statement
- optionally one block-preservation lemma statement if it is clean enough TypingStep_preser